

Automata

Limita de timp: 2 seconds

Limita de memorie: 1024 MiB

Există un câmp format din N celule într-un rând, numerotate de la 0 până la $N - 1$ de la stânga la dreapta. Celulele au înălțimi distincte: înălțimea celulei i este p_i , iar șirul $p_0, p_1, \dots, p_{N-1}$ este o permutare a numerelor $0, 1, \dots, N - 1$.

Două celule cu **indicii** i și j se numesc *apropriate* dacă și numai dacă $|i - j| \leq 1$. În particular, fiecare celulă este apropiată de ea însăși.

Un robot stă pe câmp, fiind inițial plasat pe o anumită celulă. Robotul acceptă comenzi, fiecare dintre ele poate fi de următoarele două tipuri:

- **MAX**: dintre toate celulele apropiate de celula curentă a robotului, poziția următoare a robotului va fi în celula unică cu **înălțimea maximă**;
- **MIN**: dintre toate celulele apropiate de celula curentă a robotului, poziția următoare a robotului va fi în celula unică cu **înălțimea minimă**.

Deoarece o celulă este apropiată de ea însăși, o comandă poate lăsa robotul în aceeași celulă.

Un *program* este o secvență finită de comenzi, unde fiecare comandă este fie **MAX**, fie **MIN**. Dacă robotul începe în celula X și urmează programul S , se termină într-o celulă determinată în mod unic, notată cu $\text{result}(S, X)$.

Veți primi Q interogări, fiecare dintre ele dându-vă o mulțime de celule de pornire de dimensiune K_i ($0 \leq i < Q$). Mai exact, a i -a interogare va fi pentru celulele $x_0, x_1, \dots, x_{K_i-1}$; robotul va fi plasat într-una dintre ele, dar nu se știe în care în avans. Pentru fiecare interogare trebuie să determinați dacă există un program care mută robotul în aceeași celulă finală, indiferent care poziție de pornire din x va fi folosită. Formal, trebuie să determinați dacă există un program S astfel încât:

$$\text{result}(S, x_0) = \text{result}(S, x_1) = \dots = \text{result}(S, x_{K_i-1}).$$

Permutarea p este **aceeași** pentru toate interogările.

Rețineți că nu trebuie să construiți un asemenea program – doar să raportați dacă există.

Detalii de implementare

Prima funcție pe care trebuie să o implementați este:

```
void initialize(std::vector<int> p)
```

- p : o permutare a numerelor de la 0 la $N - 1$.

A doua funcție pe care trebuie să o implementați este:

```
bool exists_program(std::vector<int> x)
```

- x : un tablou care specifică celulele date pentru o interogare, în ordine strict crescătoare.

Funcția `initialize` este apelată exact o dată, înainte de a face vreun apel al funcției `exists_program`.

Funcția `exists_program` este apelată Q ori, o dată pentru fiecare interogare. Trebuie să returneze `true` dacă există un program după care robotul ajunge în aceeași celulă indiferent din care dintre celulele date a pornit, și `false` în caz contrar.

Restricții

- $3 \leq N \leq 2 \cdot 10^5$
- $1 \leq Q \leq 5 \cdot 10^5$
- $p_0, p_1, \dots, p_{N-1}$ este o permutare a numerelor $0, 1, \dots, N - 1$
- $2 \leq K_i \leq N$
- $0 \leq x_0 < x_1 < \dots < x_{K_i-1} < N$ pentru fiecare interogare
- Suma valorilor K_i pentru toate cele Q interogări nu depășește 10^6 .

Exemple

Sample grader input 1	Sample grader output 1
3 3 0 2 1 2 0 2 3 0 1 2 2 1 2	111

Sample grader input 2	Sample grader output 2
7 3 0 4 2 1 3 5 6 3 0 1 3 3 1 3 4 2 3 6	001

Explicația exemplului 1

Pentru acest exemplu, $p = [0, 2, 1]$. Pentru a doua interogare, robotul poate să înceapă în celula 0, celula 1 sau celula 2. Consideră programul `[MAX]`.

- Când robotul începe în celula 0: celulele apropiate de 0 sunt celulele cu indicii 0 și 1. Când robotul execută comanda `MAX`, deoarece $p_0 < p_1$, se mută în celula 1.
- Când robotul începe în celula 1: celulele apropiate de 1 sunt celulele cu indicii 0, 1 și 2. Când robotul execută comanda `MAX`, deoarece $p_0 < p_1$ și $p_1 > p_2$, rămâne în celula 1.
- Când robotul începe în celula 2: celulele apropiate de 2 sunt celulele cu indicii 1 și 2. Când robotul execută comanda `MAX`, deoarece $p_1 > p_2$, se mută în celula 1.

Prin urmare, există un program după care robotul ajunge în celula 1 din toate trei celulele de start, și răspunsul pentru această interogare este `true`. Mai există și alte programe care funcționează, cum ar fi `[MIN, MAX]`, `[MIN, MIN, MAX, MIN, MAX, MIN]` și `[MAX, MIN, MAX]`.

Explicația exemplului 2

Pentru acest exemplu, $p = [0, 4, 2, 1, 3, 5, 6]$.

Pentru primele două interogări, se poate arăta că nu există niciun program pentru care robotul să ajungă în aceeași celulă din fiecare poziție de pornire dată, deci răspunsul pentru ambele este `false`.

Luați în considerare ultima interogare, pentru care robotul poate porni din celula 3 sau din celula 6, și luați în considerare programul `[MAX, MAX, MAX]`.

- Când robotul pornește din celula 3: celulele apropiate de 3 sunt celulele cu indicii 2, 3 și 4. Deoarece $p_3 < p_4$ și $p_2 < p_4$, robotul se mută în celula 4. Efectuând următoarele două comenzi în același mod, ajunge în celula 6.
- Când robotul pornește din celula 6, rămâne în celula 6 pe tot parcursul programului și se află la final tot acolo.

Prin urmare, răspunsul pentru această interogare este `true`. Programul `[MIN, MAX, MAX, MAX]` funcționează de asemenea. Cu toate acestea, alte programe ar putea să nu funcționeze. De exemplu, dacă programul este `[MAX, MAX]`, robotul ajunge în celula 5 când pornește din celula 3, dar ajunge în celula 6 când pornește din celula 6.

Subtask-uri

Subtask	Puncte	N	Q	K_i	Restricții suplimentare
0	0	-	-	-	Exemplele.
1	3	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$= 2$	Dacă răspunsul pentru o interogare este pozitiv, atunci există un program valid care utilizează exact 1 comandă.
2	7				Dacă răspunsul pentru o interogare este pozitiv, atunci există un program valid care utilizează cel mult 5 comenzi.
3	9	≤ 100	≤ 500	$= 2$	-
4	17	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$= 2$	-
5	7	$\leq 2 \cdot 10^5$			$x_1 = x_0 + 1$ pentru fiecare interogare.
6	8	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$\leq N$	-
7	13	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$\leq N$	La început permutarea este crescătoare, apoi descrescătoare și apoi din nou crescătoare. Formal, există doi întregi a, b astfel încât $0 < a < b < N - 1$ și $p_0 < p_1 < \dots < p_a > p_{a+1} > \dots > p_b < p_{b+1} < \dots < p_{N-1}$.
8	13				$p_0 < p_1 > p_2 < p_3 > \dots < p_{N-1}$ dacă N este par, sau $p_0 < p_1 > p_2 < p_3 > \dots > p_{N-1}$ if N este impar.
9	23				-

Sample grader

Formatul de intrare este următorul:

- linia 1: două numere întregi – valorile N și Q ;
- linia 2: N numere întregi $p_0, p_1, \dots, p_{N-1}$, unde p_i este înălțimea celulei a i -a;
- linia $3 + i$: celulele date pentru interogarea i -a – un număr întreg K_i , urmat pe aceeași linie de K_i numere întregi $x_0, x_1, \dots, x_{K_i-1}$, celulele de pornire posibile ale robotului.

Formatul de ieșire este următorul:

- linia 1: un șir binar de lungime Q , caracterul i din șir este 1 dacă răspunsul pentru interogarea i -a este `true`, și 0 în caz contrar.